

GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Caso base	Importar Reportes Operativos Bitakor con campos ID Reporte, Fecha Emisión, Tipo Reporte, Descripción, ID Empleado (RF-01), Zona
Operaciones	Enviar Reporte Operativo, Extraer Datos del Reporte, Registrar Datos del Reporte
Referencia base	01_Guia_PlantUML_CU_Analisis_TeKMess_V1.0 (RF-03)

1. Propósito de la guía

Esta guía complementa el modelado inicial de casos de uso del sistema TekMess. Su finalidad es demostrar cómo el requisito funcional RF-03: Importar Reportes Operativos Bitakor (que permite a los guardias y supervisores importar reportes operativos generados en el sistema Bitakor hacia el sistema SNAAR) puede analizarse de forma progresiva mediante tres modelos técnicos conectados: casos de uso de análisis, clases de análisis y diagramas de secuencia de análisis. El énfasis de este documento no es la programación de la interfaz, sino la comprensión de las responsabilidades de cada elemento del modelo (Frontera, Control y Entidad) y asegurar la trazabilidad completa desde la necesidad expresada por el Guardia o Supervisor en la ERS hasta la interacción detallada entre los objetos del sistema, incluyendo la dependencia con RF-01 (Gestionar Empleados) para validar la identidad del emisor del reporte.

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesitan hacer el Guardia y el Supervisor para importar reportes desde Bitakor?	Diagrama de casos de uso y matriz de trazabilidad RF-CU	Permite identificar las responsabilidades del sistema y definir las clases frontera, control y entidad necesarias para cumplir con el RF-03.
Clases de análisis	¿Qué objetos participan internamente para importar y registrar un reporte operativo?	Diagrama de clases con estereotipos: boundary (interfaz), control (lógica), entity (datos del reporte) y repository (persistencia en PostgreSQL)	Define los objetos y métodos específicos que aparecerán e interactuarán en los diagramas de secuencia.
Secuencias de análisis	¿Cómo colaboran el actor y los objetos para ejecutar la importación de un reporte Bitakor?	Diagramas de secuencia por operación (Enviar, Extraer Datos, Registrar Datos)	Valida que el flujo de mensajes sea coherente con los requisitos antes de la implementación en Dart/Flutter.

3. Caso base actualizado: Importar Reportes Operativos Bitakor (TekMess)

El módulo se basa en una interfaz denominada "Importar Reporte Bitakor", diseñada para que el Guardia o Supervisor registre en SNAAR los reportes operativos generados en el sistema externo Bitakor.

Dato	Tipo sugerido	Descripción	Validación mínima
ID Reporte	Texto corto	Identificador único del reporte en Bitakor	Obligatorio, no debe existir previamente en la BD.
Fecha Emisión	Fecha/Hora	Fecha y hora de generación del reporte en Bitakor	Obligatorio.
Tipo de Reporte	Enum / Lista	Categoría: INCIDENTE, RONDA, NOVEDAD, EMERGENCIA	Obligatorio, debe pertenecer a uno de los 4 tipos definidos.
Descripción	Texto largo	Detalle del evento reportado en campo	Obligatorio.
ID Empleado	Texto corto	Cédula del Guardia/Supervisor emisor del reporte	Obligatorio, debe existir en RepositorioEmpleado (trazabilidad RF-01).
Zona	Texto	Área o sector donde ocurrió el evento	Obligatorio.

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Enviar Reporte	Enviar Reporte Operativo	Captura el archivo Bitakor, valida su formato, extrae los datos y verifica que el emisor sea un empleado activo (RF-01).
Ver Vista Previa	Extraer Datos del Reporte	Parsea el archivo Bitakor y muestra al usuario los datos que serán importados antes de confirmar el registro.
Confirmar Importación	Registrar Datos del Reporte	Persiste el ReporteOperativo en la base de datos PostgreSQL tras la confirmación del usuario.
Cancelar	—	Descarta el proceso de importación y regresa a la pantalla anterior.

4. Requisitos funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-03.01	El sistema debe permitir al Guardia/Supervisor enviar un reporte operativo desde Bitakor.	Guardia / Supervisor	Enviar Reporte Operativo
RF-03.02	El sistema debe extraer automáticamente los datos del archivo Bitakor recibido.	Sistema	Extraer Datos del Reporte
RF-03.03	El sistema debe validar que el formato del reporte sea compatible con SNAAR.	Sistema	Validar Formato del Reporte
RF-03.04	El sistema debe registrar los datos extraídos en la base de datos PostgreSQL.	Sistema	Registrar Datos del Reporte

RF-03.05	El sistema debe verificar que el emisor del reporte sea un empleado activo registrado mediante RF-01.	Sistema	Validar Emisor (RF-01)
----------	---	---------	------------------------

5. Diagrama de casos de uso de análisis

Este diagrama representa las funciones visibles para el Guardia y el Supervisor dentro del módulo Importar Reportes Operativos Bitakor de TekMess. Siguiendo la estructura de la guía, la validación del formato y del emisor se modela como comportamiento obligatorio mediante relaciones de inclusión (<<include>>).

Código PlantUML sugerido

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Guardia" as Guardia
actor "Supervisor" as Supervisor
actor "Sistema Bitakor" as Bitakor

rectangle "Sistema TekMess - Módulo Importar Reportes Operativos Bitakor" {
    usecase "Enviar Reporte Operativo" as UC1
    usecase "Extraer Datos del Reporte" as UC2
    usecase "Validar Formato del Reporte" as UC3
    usecase "Registrar Datos del Reporte" as UC4
}

Guardia --> UC1
Supervisor --> UC1
Bitakor --> UC1

UC1 ..> UC3 : <<include>>
UC1 ..> UC2 : <<include>>
UC2 ..> UC4 : <<include>>

note right of UC3
Validaciones críticas:
- Formato Bitakor válido
- Campos obligatorios presentes
- ID de reporte único
- Emisor registrado en el sistema (RF01)
end note

@enduml
```

6. Derivación de clases de análisis

A partir de los casos de uso del módulo RF-03: Importar Reportes Operativos Bitakor, identificamos las clases de análisis utilizando los estereotipos de frontera, control, entidad y repositorio. Esta separación asegura que la lógica de negocio esté aislada de la interfaz y de la base de datos PostgreSQL.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con RF-03
Boundary / Frontera	Recibe el archivo Bitakor del Guardia/Supervisor y muestra resultados de la importación.	FormularioImportarReporte	Representa la pantalla con selector de archivo y botones de acción.
Control	Coordina la validación del formato, la extracción de datos y la verificación del emisor (RF-01).	ControlReporteOperativo	Valida formato Bitakor, parsea campos y gestiona el flujo de importación.
Entity / Entidad	Representa la información del reporte operativo en el sistema.	ReporteOperativo	Contiene: idReporte, fechaEmision, tipoReporte, descripcion, idEmpleado, zona.
Repositorio / Colección	Conserva y recupera los reportes en la base de datos.	RepositorioReporteOperativo	Permite guardar, buscar por ID, verificar duplicados y listar todos los reportes.
Repositorio RF-01 (trazabilidad)	Valida que el emisor del reporte sea un empleado registrado.	RepositorioEmpleado (RF-01)	Consulta existeCedula(idEmpleado) para verificar trazabilidad con RF-01.

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "FormularioImportarReporte" <<boundary>> {
    - txtArchivoReporte: String
    - lblEstadoImportacion: String
    - cbTipoReporte: TipoReporte
    + clickEnviarReporte()
    + clickCancelar()
    + mostrarMensaje(msg: String)
    + mostrarDatosExtraidos(reporte: ReporteOperativo)
}

class "ControlReporteOperativo" <<control>> {
    + enviarReporte(archivo): Resultado
    + extraerDatos(archivo): ReporteOperativo
    + validarFormato(archivo): boolean
    + validarEmisor(idEmpleado): boolean
    + registrarReporte(reporte): Resultado
}
```

```

}

class "ReporteOperativo" <<entity>> {
    - idReporte: String
    - fechaEmision: Date
    - tipoReporte: TipoReporte
    - descripcion: String
    - idEmpleado: String
    - zona: String
    + getDatos(): ReporteOperativo
}

enum "TipoReporte" {
    INCIDENTE
    RONDA
    NOVEDAD
    EMERGENCIA
}

class "RepositorioReporteOperativo" <<repository>> {
    + existeReporte(idReporte): boolean
    + guardar(ReporteOperativo): void
    + buscarPorId(idReporte): ReporteOperativo
    + listarTodos(): List<ReporteOperativo>
}

class "RepositorioEmpleado" <<repository>> {
    + existeCedula(cedula): boolean
    + buscarPorCedula(cedula): Empleado
}

FormularioImportarReporte --> ControlReporteOperativo : solicita operación
ControlReporteOperativo --> ReporteOperativo : extrae/crea entidad
ControlReporteOperativo --> RepositorioReporteOperativo : gestiona persistencia en PostgreSQL
ControlReporteOperativo --> RepositorioEmpleado : valida emisor (RF01)
RepositorioReporteOperativo o-- "0..*" ReporteOperativo : almacena
@enduml

```

7. Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
-------------	----------	---------	----------------------	--------------------------------

Enviar Reporte Operativo	Captura el archivo Bitakor del Guardia/Supervisor.	Valida formato, extrae datos y coordina la importación completa, incluyendo la verificación del emisor en RF-01.	Crea la entidad ReporteOperativo y persiste en RepositorioReporteOperativo; valida emisor en RepositorioEmpleado (RF-01).	Seleccionar archivo → Validar → Extraer → Verificar emisor (RF-01) → Crear entidad → Guardar → Confirmar.
Extraer Datos del Reporte	Muestra vista previa de datos extraídos al usuario.	Parsea el archivo Bitakor y construye el objeto ReporteOperativo.	Crea el objeto ReporteOperativo con todos sus atributos a partir del archivo.	Enviar archivo → Validar formato → Parsear campos → Construir objeto → Mostrar vista previa.
Registrar Datos del Reporte	Muestra confirmación de importación exitosa y lista actualizada.	Verifica duplicados y coordina el almacenamiento en la base de datos.	Persiste el ReporteOperativo en RepositorioReporteOperativo (PostgreSQL).	Confirmar → Verificar duplicado → Guardar → Confirmar éxito → Actualizar lista.

8. Diagramas de secuencia de análisis

Estos diagramas muestran la colaboración dinámica de los objetos de TekMess a lo largo del tiempo para cumplir con las operaciones del RF-03. El diagrama 8.1 es el flujo principal e incluye la trazabilidad directa con RF-01 mediante la consulta al RepositorioEmpleado.

8.1 Secuencia: Enviar Reporte Operativo (flujo principal con trazabilidad RF-01)

```
@startuml
title Secuencia de análisis - Enviar Reporte Operativo (TekMess)

actor "Guardia / Supervisor" as Actor
boundary "FormularioImportarReporte" as UI
control "ControlReporteOperativo" as Ctrl
database "RepositorioEmpleado" as RepoEmp
database "RepositorioReporteOperativo" as Repo
entity "ReporteOperativo" as Rep

Actor -> UI : Selecciona archivo de reporte Bitakor
Actor -> UI : Clic en Enviar Reporte
UI -> Ctrl : enviarReporte(archivoReporte)
Ctrl -> Ctrl : validarFormato(archivoReporte)
Ctrl -> Ctrl : extraerDatos(archivoReporte)
Ctrl -> RepoEmp : existeCedula(idEmpleado)
RepoEmp --> Ctrl : true (emisor registrado en RF01)
Ctrl -> Repo : existeReporte(idReporte)
```

```
Repo --> Ctrl : false (no existe)

Ctrl -> Rep : new ReporteOperativo(idReporte, fechaEmision, tipoReporte, descripcion, idEmpleado, zona)

Ctrl -> Repo : guardar(Rep)

Repo --> Ctrl : Confirmación PostgreSQL exitosa

Ctrl --> UI : Importación exitosa

UI --> Actor : Muestra mensaje de confirmación y datos importados

@enduml
```

8.2 Secuencia: Extraer Datos del Reporte Operativo

```
@startuml

title Secuencia de análisis - Extraer Datos del Reporte Operativo (TekMess)

actor "Guardia / Supervisor" as Actor
boundary "FormularioImportarReporte" as UI
control "ControlReporteOperativo" as Ctrl
entity "ReporteOperativo" as Rep

Actor -> UI : Envía archivo de reporte Bitakor
UI -> Ctrl : extraerDatos(archivoReporte)
Ctrl -> Ctrl : validarFormato(archivoReporte)
Ctrl -> Rep : new ReporteOperativo()
Ctrl -> Rep : parsear(idReporte, fechaEmision, tipoReporte, descripcion, idEmpleado, zona)
Rep --> Ctrl : Objeto ReporteOperativo construido
Ctrl --> UI : Datos extraídos correctamente
UI --> Actor : Muestra vista previa de datos extraídos del reporte

@enduml
```

8.3 Secuencia: Registrar Datos del Reporte Operativo

```
@startuml

title Secuencia de análisis - Registrar Datos del Reporte Operativo (TekMess)

actor "Guardia / Supervisor" as Actor
boundary "FormularioImportarReporte" as UI
control "ControlReporteOperativo" as Ctrl
database "RepositorioReporteOperativo" as Repo
entity "ReporteOperativo" as Rep

Actor -> UI : Confirma importación del reporte
UI -> Ctrl : registrarReporte(Rep)
Ctrl -> Repo : existeReporte(idReporte)
Repo --> Ctrl : false (no existe)
Ctrl -> Repo : guardar(Rep)
Repo --> Ctrl : Registro exitoso en PostgreSQL
Ctrl --> UI : Resultado exitoso
```

UI --> Actor : Muestra confirmación y lista de reportes actualizada

@enduml

9. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Enviar Reporte Operativo	El usuario debe tener sesión activa y el archivo de reporte Bitakor debe estar disponible.	Seleccionar archivo → Validar formato → Extraer datos → Verificar emisor en RF-01 → Crear entidad → Guardar en PostgreSQL → Generar confirmación.	Reporte importado exitosamente y disponible para auditoría en SNAAR.
Extraer Datos del Reporte	El archivo debe tener formato Bitakor compatible con SNAAR.	Parsear campos del archivo → Verificar integridad de los datos → Construir objeto ReporteOperativo.	Datos del reporte estructurados y disponibles para su registro en la base de datos.
Registrar Datos del Reporte	El reporte no debe existir previamente en la base de datos (sin duplicados).	Verificar duplicado por ID Reporte → Guardar en PostgreSQL → Notificar éxito.	Reporte persistido correctamente en la base de datos PostgreSQL.

10. Conclusión técnica

La implementación de la metodología de análisis para el módulo Importar Reportes Operativos Bitakor de TekMess demuestra que la relación entre los casos de uso, las clases de análisis y los diagramas de secuencia permite verificar que el sistema no se construye como un conjunto de piezas aisladas. En este proceso de modelado, los casos de uso definen qué acciones funcionales requieren el Guardia y el Supervisor; las clases de análisis (Frontera, Control, Entidad y Repositorio) establecen qué objetos colaboran internamente; y los diagramas de secuencia ilustran la interacción dinámica necesaria para cumplir con cada operación de importación. La trazabilidad con RF-01 (Gestionar Empleados) se materializa mediante la consulta al RepositorioEmpleado durante el flujo de envío del reporte, garantizando que solo empleados activos registrados en el sistema puedan ser emisores de reportes operativos. Esta dependencia técnica asegura la integridad referencial entre módulos y prepara el camino para un desarrollo de software ordenado y escalable en Dart/Flutter con persistencia en PostgreSQL.

Fuente interna de referencia

- 01_Guia_PlantUML_CU_Analisis_TekMess_V1.0 (RF-01) — estructura base y patrón de clases de análisis.
- 01_Guia_PlantUML_CU_Analisis_TekMess_V1.0 (RF-03) — referencia del módulo actual.